

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет програмних систем та прикладної математики
Кафедра програмного забезпечення комп’ютерних систем




**Комп’ютерний практикум №2**
з дисципліни **“Програмне забезпечення технології цифрових двійників”**
тема **“Конвеєр обробки потокових даних та синхронізація стану цифрового двійника”**

Виконав
студент V курсу
групи КП-61мн
Семенюк Сергій Іванович

Київ 2026

**Тема:** розроблення конвеєра обробки потокових даних та програмних компонентів синхронізації стану цифрового двійника в реальному часі.

**Мета:** набути практичних умінь розроблення методів та алгоритмів обробки даних у реальному часі для синхронізації фізичних об’єктів з їхніми цифровими копіями.

**Завдання:** розробити програмні компоненти, які приймають потік телеметрії, виконують її оброблення, підтримують актуальну модель стану цифрового двійника та зберігають історію станів у сховищі даних часових рядів.

**Кроки реалізації:**

1. Спроєктувати модель стану цифрового двійника: перелік атрибутів стану, їх типи, правила обчислення похідних показників (агрегати, середні, індикатори режимів).
2. Реалізувати приймання потоку телеметрії з IoT-каналу (робота 2) із буферизацією повідомлень та впорядкуванням за часовими мітками.
3. Реалізувати попереднє оброблення даних: валідація, фільтрація викидів, заповнення пропусків, приведення до єдиної частоти дискретизації.
4. Розробити та реалізувати правила синхронізації: періодичне оновлення, оновлення за подією, оновлення за перевищенням порогу зміни значення.
5. Реалізувати компонент підтримання актуального стану двійника в пам’яті та механізм сповіщення підписників про зміну стану.
6. Розгорнути сховище даних часових рядів (наприклад, InfluxDB або TimescaleDB) та реалізувати збереження історії телеметрії і станів двійника.
7. Виміряти характеристики синхронізації: середню та максимальну затримку між моментом генерації вимірювання та оновленням стану двійника, пропускну здатність конвеєра.
8. Перевірити роботу конвеєра за різних сценаріїв (штатний режим, сплеск інтенсивності даних, тимчасова відсутність зв’язку); оформити звіт.

**Вимоги до змісту звіту:** модель стану цифрового двійника; схема конвеєра обробки потокових даних; опис реалізованих правил синхронізації; опис структури сховища часових рядів; результати вимірювання затримки та пропускної здатності (таблиці, графіки); знімки екрана; висновки.

**Захист роботи:** демонстрація оновлення стану цифрового двійника в реальному часі за надходження телеметрії та вибірки історичних даних зі сховища. Орієнтовні контрольні запитання: правила та шаблони синхронізації; способи оброблення пропусків і викидів у потокових даних; особливості сховищ даних часових рядів порівняно з реляційними базами даних. (КП 3. Конвеєр обробки потокових даних та синхронізація стану цифрового двійника)

## Хід роботи

**1. Спроєктувати модель стану цифрового двійника: перелік атрибутів стану, їх типи, правила обчислення похідних показників (агрегати, середні, індикатори режимів).**

Для забезпечення синхронізації цифрового двійника з фізичним об'єктом необхідно визначити структуру його поточного стану. На відміну від окремих телеметричних повідомлень, які містять значення одного сенсора на певний момент часу, модель стану цифрового двійника об'єднує інформацію про всі компоненти електроскутера та забезпечує цілісне представлення його поточного стану.

Спираючись на перелік телеметричних сигналів, визначений у попередніх комп'ютерних практикумах, розробимо модель стану цифрового двійника електроскутера. Вона міститиме три основні групи атрибутів:

- Первинні атрибути – останні отримані значення телеметричних сигналів від сенсорів.
- Похідні атрибути – показники, які обчислюються на основі отриманих телеметричних даних.
- Службові атрибути – інформація про актуальність даних, час останнього оновлення та стан зв'язку з джерелом телеметрії.

### 1.1. Визначення первинних атрибутів стану

Первинні атрибути відповідають восьми телеметричним сигналам, які вже використовуються в розробленому емуляторі сенсорів. Для кожного атрибута визначимо тип даних, одиницю вимірювання та частоту оновлення.

| № | Атрибут | Тип даних | Одиниця вимірювання | Частота оновлення | Джерело |
|---|---|---|---|---|---|
| 1 | speed | float | км/год | 10 Гц | SIG_VEL_01 |
| 2 | motor_rpm | float | об/хв | 10 Гц | SIG_RPM_02 |
| 3 | battery_current | float | A | 1 Гц | SIG_CUR_03 |
| 4 | battery_voltage | float | B | 1 Гц | SIG_VOL_04 |
| 5 | motor_temperature | float | °C | 0,1 Гц | SIG_TMP_05 |
| 6 | battery_soc | float | % | 0,1 Гц | SIG_SOC_06 |
| 7 | braking | bool | – | Подієво | SIG_BRK_07 |
| 8 | anti_theft_lock | bool | – | Подієво | SIG_LCK_08 |

Значення цих атрибутів оновлюватимуться після отримання відповідних MQTT-повідомлень. Наприклад, після отримання повідомлення із сенсора SIG_VEL_01 атрибут speed набуватиме значення швидкості, зазначеного в полі value.

Для кожного первинного атрибута передбачено збереження часової мітки останнього отриманого вимірювання. Це дасть змогу незалежно контролювати актуальність кожного телеметричного параметра, оскільки різні сенсори передають дані з різною частотою. Загальний атрибут last_update відображатиме час останньої зміни моделі стану цифрового двійника.

### 1.2. Визначення похідних атрибутів

Окрім безпосередньо отриманих телеметричних значень, модель стану цифрового двійника міститиме похідні показники. Вони обчислюватимуться на основі первинних атрибутів і використовуватимуться для узагальнення поточного стану електроскутера.

| Атрибут | Тип даних | Правило обчислення | Призначення |
|---|---|---|---|
| average_speed | float | Середнє значення швидкості за визначене часове вікно | Аналіз середньої швидкості руху |
| battery_power | float | Добуток напруги та струму батареї | Оцінювання потужності споживання |
| trip_mode | string | Визначення за швидкістю та станом гальмування | Визначення поточного режиму руху |
| emergency_state | bool | Перевірка критичних значень температури, струму та напруги | Виявлення аварійного стану |

Середня швидкість руху обчислюватиметься за останніми вимірюваннями швидкості в межах заданого часового вікна:

$$v_{avg} = \frac{1}{N}\sum_{i=1}^{N} v_i$$

де $v_i$ – окреме вимірювання швидкості, а $N$ – кількість вимірювань у вибраному часовому вікні.

Потужність споживання електроенергії визначатиметься за формулою:

$$P = U \cdot I$$

де $U$ – напруга акумуляторної батареї (В), а $I$ – струм батареї (А). Результат вимірюватиметься у ватах.

Поточний режим руху визначатиметься на основі швидкості електроскутера, стану гальмування та зміни швидкості порівняно з попереднім вимірюванням. Для цього передбачено чотири можливі значення атрибута trip_mode:

- idle – швидкість близька до нуля;
- braking – активовано сигнал гальмування;
- accelerating – швидкість зростає порівняно з попереднім вимірюванням;
- cruise – електроскутер рухається без активного гальмування, а швидкість залишається приблизно сталою.

Для визначення режиму розгону використовуватиметься порівняння поточного та попереднього значень швидкості. Щоб уникнути помилкового перемикання режимів через незначні коливання показників сенсора, під час порівняння може враховуватися допустимий поріг зміни швидкості.

Аварійний стан визначатиметься на основі перевірки критичних значень телеметричних параметрів. Зокрема, враховуватимуться перевищення допустимої температури двигуна, надмірний струм акумуляторної батареї та критичне зниження її напруги. Конкретні порогові значення будуть визначені під час реалізації компонента оброблення телеметрії.

Важливо розрізняти режим руху та режим обмеження максимальної швидкості. Перший визначається за поточними показниками телеметрії, тоді як другий відповідає налаштуванню системи керування електроскутером.

### 1.3. Службові атрибути моделі стану

Для контролю синхронізації та актуальності телеметричних даних передбачимо додаткові службові атрибути.

| Атрибут | Тип даних | Призначення |
|---|---|---|
| last_update | datetime | Час останнього оновлення моделі стану |
| connection_status | string | Поточний стан підключення джерела телеметрії |
| data_valid | bool | Ознака валідності отриманих даних |
| active_anomalies | array of string | Ідентифікатор активного відхилення, якщо воно виявлене |

Атрибут connection_status відображатиме доступність джерела телеметрії. Для цього можна використати вже реалізований у КП2 механізм Last Will and Testament: повідомлення online означатиме активне підключення, а offline – нештатне від'єднання джерела даних.

Атрибут data_valid використовуватиметься для позначення результату перевірки отриманих повідомлень. Якщо повідомлення не пройшло валідацію, його значення не повинно безпосередньо змінювати актуальний стан цифрового двійника.

Атрибут active_anomalies міститиме перелік ідентифікаторів виявлених відхилень, наприклад overheat, overcurrent або voltage_drop. Якщо активних відхилень немає, масив буде порожнім.

### 1.4. Загальна структура моделі стану

Таким чином, модель стану цифрового двійника електроскутера складатиметься з первинних, похідних і службових атрибутів. Її узагальнену структуру можна представити таким чином:


```
{
 "object_id": "scooter_01",
 "telemetry": {
  "speed": 32.5,
  "motor_rpm": 1787.5,
  "battery_current": 9.2,
  "battery_voltage": 49.8,
  "motor_temperature": 42.5,
  "battery_soc": 89.5,
  "braking": false,
  "anti_theft_lock": false
 },
 "derived": {
  "average_speed": 30.8,
  "battery_power": 458.16,
  "trip_mode": "cruise",
  "emergency_state": false
 },
 "metadata": {
  "last_update": "2026-11-01T10:30:15.200000+00:00",
  "connection_status": "online",
  "data_valid": true,
  "active_anomalies": ["overheat", "overcurrent"]
 }
}
```


Наведений JSON демонструє узагальнену структуру моделі стану цифрового двійника. Для спрощення прикладу часові мітки окремих телеметричних атрибутів не відображено. Під час реалізації моделі для

кожного первинного атрибута додатково зберігатиметься час отримання останнього вимірювання у форматі ISO 8601. Це дасть змогу контролювати актуальність окремих параметрів незалежно від загального часу оновлення моделі, який відображатиметься атрибутом last_update.

Отже, на першому етапі визначено структуру моделі стану цифрового двійника електроскутера, яка охоплює всі телеметричні сигнали з попереднього практикуму, додаткові обчислювані показники та службову інформацію. Така модель стане основою для подальшої реалізації конвеєра оброблення потокових даних, синхронізації стану в пам'яті та збереження історії вимірювань.

**2. Реалізувати приймання потоку телеметрії з IoT-каналу (робота 2) із буферизацією повідомлень та впорядкуванням за часовими мітками.**

Для забезпечення надходження телеметричних даних до конвеєра оброблення розроблено програмний компонент stream_receiver.py, який підключається до MQTT-брокера та приймає повідомлення від емулятора сенсорів, реалізованого в попередньому комп'ютерному практикумі.

Для реалізації цього етапу створено два програмні файли:

- stream_receiver.py – компонент приймання MQTT-повідомлень, їх декодування та передавання до буфера;
- message_buffer.py – компонент буферизації та впорядкування повідомлень за часовими мітками.

Для генерації та передавання телеметрії продовжують використовуватися файли emulator.py та emulator_publisher.py, розроблені в КП2. Таким чином, новий компонент інтегрується з наявним IoT-каналом без необхідності змінювати формат повідомлень.

**2.1. Реалізація буфера повідомлень**

У файлі message_buffer.py реалізовано клас MessageBuffer, призначений для тимчасового накопичення отриманих телеметричних повідомлень та їх впорядкування за часовими мітками.

Для зберігання повідомлень використовується структура даних типу мінімальної купи (heapq), яка дає змогу отримувати повідомлення з найранішою часовою міткою серед тих, що перебувають у буфері.

Під час ініціалізації буфера задається параметр reorder_window, який визначає часове вікно накопичення повідомлень. У поточній реалізації його значення становить 0,5 секунди. Це дає змогу накопичувати

повідомлення, які можуть надходити з невеликим порушенням порядку, перш ніж передавати їх на подальше оброблення.

Клас MessageBuffer містить два основні методи:

- add() – додає отримане повідомлення до буфера, використовуючи його часову мітку для впорядкування;
- get_ready() – повертає повідомлення, які можна передавати на подальше оброблення після завершення часового вікна або досягнення відповідної умови впорядкування.

Додатково реалізовано механізм виявлення запізнілих повідомлень. Якщо часова мітка повідомлення є меншою за часову мітку останнього вже переданого повідомлення, воно позначається ознакою late=True. Це дає змогу ідентифікувати повідомлення, які надійшли із запізненням і не були впорядковані в межах установленого часового вікна.

### 2.2. Реалізація компонента приймання телеметрії

У файлі stream_receiver.py реалізовано підключення до MQTT-брокера Mosquitto за адресою localhost та портом 1883.

Для отримання телеметричних повідомлень використовується підписка на топік:

*object/scooter_01/sensor/#*

Символ # дає змогу отримувати повідомлення з усіх дочірніх топіків, що відповідають окремим сенсорам електроскутера. Також передбачено підписку на топік стану джерела телеметрії:

*object/scooter_01/status*

Це дає змогу отримувати інформацію про доступність джерела даних, зокрема повідомлення online та offline, які використовуються в механізмі Last Will and Testament, реалізованому в КП2

Після отримання MQTT-повідомлення компонент виконує такі операції:

1. Декодує отримані дані з формату UTF-8 та перетворює JSON-повідомлення на структуру Python.
2. Перевіряє, чи є повідомлення JSON-об'єктом та чи містить воно часову мітку у вигляді рядка.
3. Перетворює часову мітку на об'єкт datetime та приводить її до часового поясу UTC.
4. Фіксує час отримання повідомлення за допомогою атрибута received_at.

5. Передає повідомлення до проміжної черги, звідки воно надходить до MessageBuffer.

Для розділення приймання повідомлень та їх подальшого опрацювання використовується окремий потік виконання (threading). Це дає змогу не блокувати обробник MQTT-повідомлень під час роботи з буфером.

Після отримання повідомлень із буфера компонент виводить їхню структуру в консоль та записує інформацію до журналу роботи. Повідомлення, які надійшли із запізненням, додатково позначаються відповідним попередженням.

На цьому етапі реалізовано первинну перевірку структури повідомлення та часової мітки. Повна перевірка типів і діапазонів значень телеметричних параметрів, фільтрація викидів та оброблення пропусків передбачені в наступному етапі роботи.

### 2.3. Запуск та перевірка роботи компонента приймання телеметрії

Для перевірки розробленого компонента використовуються MQTT-брокер Mosquitto та емулятор сенсорів із видавцем повідомлень, реалізовані в попередньому комп'ютерному практикумі.

Перед запуском програмних компонентів необхідно переконатися, що MQTT-брокер Mosquitto працює. У Windows PowerShell це можна перевірити командою:

*Get-Service *mosquitto**

Якщо служба має статус Running, брокер уже працює, тому повторно запускати його не потрібно. Якщо служба зупинена (Stopped), її можна запустити командою:

Start-Service -Name mosquitto

За потреби службу можна зупинити командою:

*Stop-Service -Name mosquitto*

Альтернативним способом є запуск брокера безпосередньо з виконуваного файлу Mosquitto в режимі докладного виведення повідомлень:

*& "D:\Programs\Mosquitto\mosquitto.exe" -v*

У цьому випадку брокер працює безпосередньо у вікні PowerShell, де відображаються повідомлення про підключення клієнтів та обмін MQTT-повідомленнями. Для зупинки такого процесу використовується комбінація клавіш Ctrl + C.

У разі ручного запуску брокера службу Windows Mosquitto попередньо потрібно зупинити, щоб уникнути конфлікту через використання одного порту.

Після запуску MQTT-брокера в першому вікні PowerShell запускається емулятор із видавцем телеметрії:

*cd D:\dt-software\src*

*python emulator_publisher.py*

У другому вікні PowerShell запускається розроблений компонент приймання потокових даних:

*cd D:\dt-software\src*

*python stream_receiver.py*

Окремо запускати message_buffer.py не потрібно, оскільки його клас MessageBuffer імпортується безпосередньо у файлі stream_receiver.py.

Після запуску програмних компонентів було перевірено передавання телеметричних повідомлень від емулятора через MQTT-брокер до stream_receiver.py. У консолі відображалися отримані повідомлення з відповідними назвами сенсорів, значеннями вимірювань, одиницями вимірювання та часовими мітками.

```
PS D:\dt-software\src> python stream_receiver.py
Запуск компонента приймання телеметрії...
З'єднання з MQTT-брокером встановлено.
Підписка на телеметрію: object/scooter_01/sensor/#
Підписка на стан джерела: object/scooter_01/status

[СТАН ДЖЕРЕЛА] online

[ОТРИМАНО] object/scooter_01/sensor/SIG_VEL_01
{
  "sensor_id": "SIG_VEL_01",
  "name": "Швидкість",
  "value": 9.51,
  "unit": "км/год",
  "timestamp": "2026-11-01T10:07:13.698978+00:00",
  "received_at": "2026-11-01T10:07:13.699511+00:00"
}
```

Рис. 2.1. Приймання телеметричних повідомлень компонентом stream_receiver.py.

Для контролю роботи компонента також передбачено ведення журналу подій stream_receiver.log. Він створюється автоматично під час запуску stream_receiver.py, якщо ще не існує, у поточному робочому каталозі. До нього записуються відомості про отримані телеметричні повідомлення, стан підключення джерела даних та помилки приймання.

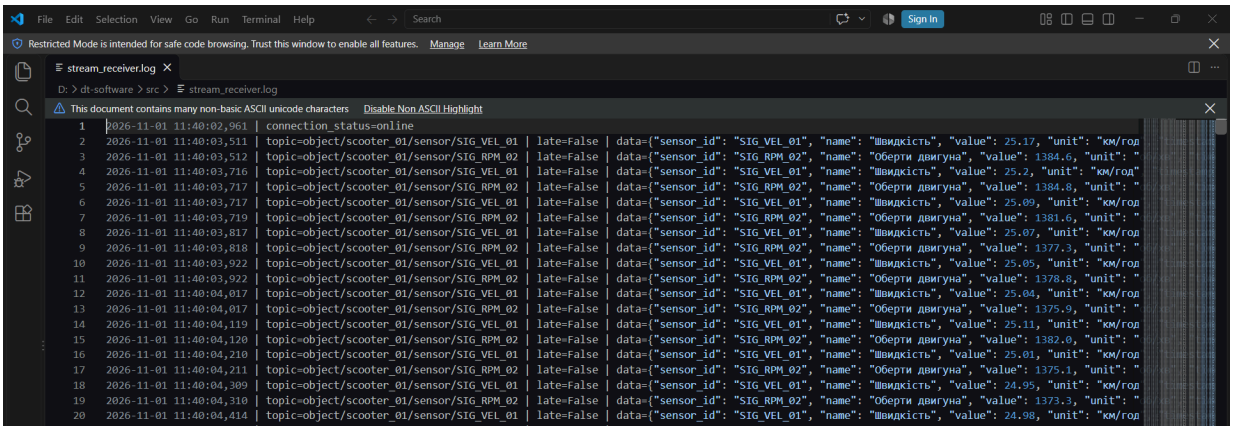



Рис.2.2. Журнал отриманих повідомлень stream_receiver.log.

Таким чином, у результаті перевірки підтверджено встановлення з'єднання з MQTT-брокером та успішне приймання телеметричних повідомлень із подальшим передаванням до буфера. Отримані дані містять часові мітки формування та приймання, що створює основу для подальшого контролю затримки й синхронізації стану цифрового двійника.

Примітка: наведений приклад підтверджує приймання телеметрії. Для окремого підтвердження впорядкування повідомлень, які навмисно надходять у неправильній послідовності, потрібно провести додатковий тест буфера.

**3. Реалізувати попереднє оброблення даних: валідація, фільтрація викидів, заповнення пропусків, приведення до єдиної частоти дискретизації.**

На третьому етапі розроблено компоненти попереднього оброблення телеметрії, призначені для перевірки отриманих повідомлень, виявлення неправдоподібних вимірювань, контролю актуальності даних і формування узгоджених наборів телеметрії з єдиною частотою дискретизації.

Для реалізації цього етапу було створено чотири програмні модулі: data_validator.py, outlier_filter.py, missing_data_handler.py та resampler.py. Також було доповнено компонент stream_receiver.py, який забезпечує

послідовне виконання операцій попереднього оброблення після отримання повідомлень із MQTT-брокера.

### 3.1. Валідація отриманих повідомлень

Для перевірки коректності вхідних телеметричних даних розроблено модуль data_validator.py, який містить клас DataValidator.

Валідація передбачає перевірку:

- наявності обов'язкових полів повідомлення: sensor_id, name, value, unit і timestamp;
- відповідності ідентифікатора датчика одному з визначених сигналів електроскутера;
- відповідності назви та одиниці вимірювання заданій конфігурації;
- типу отриманого значення;
- допустимого діапазону значень для кожного числового сигналу;
- коректності часової мітки та наявності часового поясу.

Повідомлення, що не відповідають визначеним умовам, відхиляються та реєструються в журналі. Це запобігає передаванню некоректних значень до наступних компонентів конвеєра оброблення.

### 3.2. Фільтрація аномальних вимірювань

Для виявлення неправдоподібних змін значень телеметричних показників розроблено модуль outlier_filter.py, який містить клас OutlierFilter.

У модулі реалізовано перевірку максимально допустимої швидкості зміни значень для кожного числового сигналу. Для цього зберігаються попереднє прийняте значення показника та відповідна часова мітка. Під час отримання нового вимірювання обчислюється різниця між поточним і попереднім значеннями з урахуванням часу між ними.

Якщо швидкість зміни перевищує встановлений поріг, вимірювання вважається викидом і не передається до наступних компонентів оброблення. Для подієвих сигналів така перевірка не застосовується, оскільки вони відображають дискретні зміни стану.

Встановлені пороги враховують характер зміни показників електроскутера та особливості їх емуляції, зокрема можливі відхилення струму, напруги й температури під час моделювання аномальних ситуацій.

### 3.3. Контроль пропусків та актуальності даних

Для контролю наявності й актуальності телеметричних вимірювань розроблено модуль missing_data_handler.py, який містить клас MissingDataHandler.

Компонент зберігає останнє прийняте значення кожного сигналу разом із часовою міткою його формування. Під час формування поточного зрізу телеметрії обчислюється час, що минув від моменту отримання останнього вимірювання.

Для різних сигналів визначено окремі допустимі інтервали актуальності відповідно до їхньої частоти надходження. Якщо вимірювання відсутнє або його вік перевищує встановлену межу, значення позначається як неактуальне. Для таких даних повертається ознака valid = false.

Для подієвих сигналів, зокрема стану гальмування та протиугінного блокування, зберігається останній отриманий стан, оскільки він залишається чинним до надходження наступної події.

Таким чином, компонент дає змогу розрізняти актуальні вимірювання та дані, які не можна використовувати як достовірне відображення поточного стану об'єкта.

### 3.4. Приведення даних до єдиної частоти дискретизації

Для формування узгоджених наборів телеметричних даних розроблено модуль resampler.py, який містить клас Resampler.

У межах реалізації встановлено цільову частоту формування зрізів 10 Гц, що відповідає інтервалу 0,1 секунди. Під час кожного циклу формування зрізу компонент отримує останні доступні значення сигналів із MissingDataHandler.

Оскільки первинні сигнали надходять із різною частотою, для формування узгодженого набору використовуються останні збережені вимірювання. Для кожного показника додатково передаються час його формування, вік даних та ознака актуальності.

Такий підхід забезпечує формування наборів телеметрії з єдиною частотою оновлення без необхідності повторного надсилання фізичних вимірювань від датчиків.

### 3.5. Інтеграція компонентів попереднього оброблення

Розроблені модулі інтегровано в stream_receiver.py, який було доповнено логікою послідовного виконання операцій попереднього оброблення.

Після отримання MQTT-повідомлення воно передається до черги, а потім до буфера впорядкування. Після отримання готового повідомлення виконуються такі операції:

- Перевірка часової послідовності. Запізнілі повідомлення не використовуються для оновлення останніх значень.
- Валідація структури повідомлення, типів даних і допустимих діапазонів значень.
- Перевірка швидкості зміни показника для виявлення неправдоподібних вимірювань.
- Оновлення останнього достовірного значення відповідного сигналу.
- Формування узгодженого зрізу телеметрії з частотою 10 Гц.

Результати оброблення виводяться в консоль у скороченому вигляді, а повні сформовані зрізи записуються до журналу stream_receiver.log.

### 3.6. Перевірка роботи конвеєра попереднього оброблення

Для перевірки реалізованих компонентів було запущено MQTT-брокер, емулятор електроскутера та оновлений компонент stream_receiver.py.

Під час запуску було підтверджено встановлення з'єднання з MQTT-брокером, отримання статусу джерела online та початок формування узгоджених зрізів телеметрії з цільовою частотою 10 Гц.

На початкових етапах роботи в окремих зрізах значення деяких показників відображалися як None. Це пов'язано з тим, що відповідні вимірювання ще не надійшли до компонента оброблення. Після отримання перших повідомлень значення почали відображатися у сформованих зрізах.

Зокрема, під час перевірки було отримано значення швидкості руху, температури двигуна та рівня заряду батареї. Приклад роботи компонента наведено на рисунку 3.1.

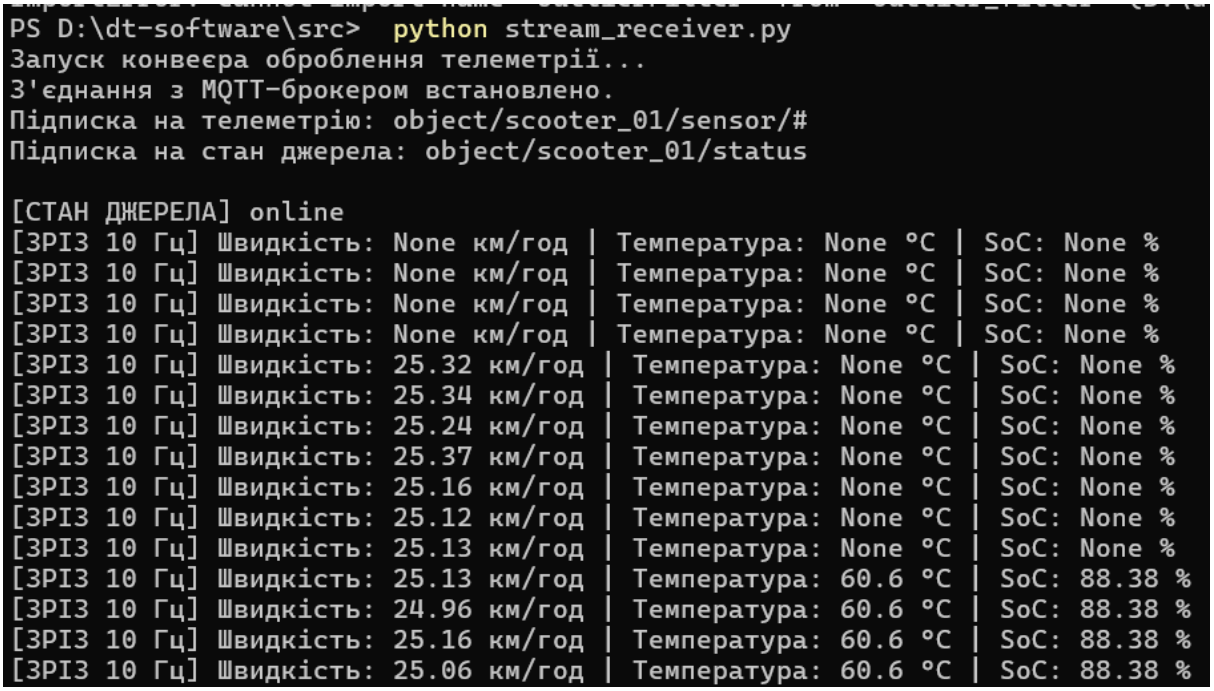



Рис. 3.1. Результат формування узгоджених зрізів телеметрії з частотою 10 Гц.

Повні результати формування зрізів також зберігаються у файлі stream_receiver.log, що дає змогу переглядати оброблені дані та використовувати їх для подальшого аналізу.

Таким чином, у результаті виконання третього етапу реалізовано попереднє оброблення потокової телеметрії, яке охоплює валідацію повідомлень, фільтрацію неправдоподібних вимірювань, контроль актуальності даних і формування узгоджених наборів із цільовою частотою 10 Гц. Розроблені компоненти інтегровано в конвеєр приймання телеметрії, що створює основу для подальшої синхронізації актуального стану цифрового двійника.

**4. Розробити та реалізувати правила синхронізації: періодичне оновлення, оновлення за подією, оновлення за перевищенням порогу зміни значення.**

**4.1. Реалізація компонента синхронізації**

Для керування оновленням стану цифрового двійника розроблено окремий модуль state_synchronizer.py, який містить клас StateSynchronizer.

Компонент призначений для визначення моментів, коли необхідно оновити цифрове представлення електроскутера на основі отриманих і попередньо оброблених телеметричних даних. Для цього реалізовано три

правила синхронізації: періодичне оновлення, оновлення за подією та оновлення за перевищенням порогу зміни значення.

Під час виконання синхронізації формується структура поточного стану цифрового двійника, яка містить:

- updated_at – час виконання синхронізації;
- sync_reason – причина оновлення стану;
- telemetry – набір актуальних телеметричних показників із часовими мітками та ознаками їхньої актуальності.

Сформований стан зберігається в атрибуті current_state класу StateSynchronizer. Також компонент веде лічильник виконаних синхронізацій sync_count і записує результати оновлення до журналу stream_receiver.log.

**4.2. Періодична синхронізація**

Першим реалізованим правилом є періодична синхронізація, яка забезпечує регулярне оновлення стану цифрового двійника незалежно від величини змін телеметричних показників.

Для цього в класі StateSynchronizer визначено параметр PERIODIC_INTERVAL, значення якого становить 1 секунду. Під час кожного циклу роботи компонента processing_worker() перевіряється час, що минув від останньої періодичної синхронізації.

Якщо встановлений інтервал минув, формується новий зріз телеметрії та виконується оновлення стану цифрового двійника. Часові інтервали вимірюються за допомогою функції time.monotonic(), що дає змогу контролювати тривалість інтервалів незалежно від змін системного часу.

Таким чином, навіть за відсутності значних змін показників цифровий двійник отримує оновлений стан щонайменше один раз на секунду за умови активної роботи конвеєра.

**4.3. Синхронізація за подією**

Другим правилом є синхронізація за подією, яка забезпечує оновлення стану цифрового двійника в разі зміни дискретних станів електроскутера.

У межах реалізації для цього використовуються два подієві сигнали:

- SIG_BRK_07 – стан гальмування;
- SIG_LCK_08 – стан протиугінного блокування.

Під час отримання повідомлення від одного з цих датчиків компонент StateSynchronizer порівнює нове значення з попереднім відомим станом відповідного сигналу. Якщо повідомлення отримано вперше або значення змінилося, виконується синхронізація.

Це дає змогу оновлювати цифровий двійник без очікування завершення періодичного інтервалу. Наприклад, під час переходу гальмівної системи зі стану неактивності до активного стану відповідна зміна відразу стає підставою для синхронізації.

**4.4. Синхронізація за перевищенням порогу зміни значення**

Третім правилом є синхронізація за порогом зміни числового показника. Її призначення полягає в оновленні цифрового двійника тоді, коли зміна певного телеметричного параметра стає достатньо значною.

Для кожного числового сигналу визначено окремий поріг зміни. Під час отримання нового вимірювання його значення порівнюється зі значенням відповідного показника, збереженим під час останньої синхронізації.

Якщо абсолютна різниця між значеннями дорівнює встановленому порогу або перевищує його, виконується оновлення стану цифрового двійника.

У таблиці 4.1 наведено встановлені порогові значення.

Таблиця 4.1

Порогові значення для синхронізації

| Ідентифікатор сигналу | Показник | Поріг зміни |
|---|---|---|
| SIG_VEL_01 | Швидкість руху | 2,0 км/год |
| SIG_RPM_02 | Частота обертання електродвигуна | 100 об/хв |
| SIG_CUR_03 | Струм акумуляторної батареї | 2,0 А |
| SIG_VOL_04 | Напруга акумуляторної батареї | 0,5 В |
| SIG_TMP_05 | Температура електродвигуна | 1,0 °C |
| SIG_SOC_06 | Рівень заряду акумуляторної батареї | 1,0 % |

Застосування окремих порогів для кожного показника дає змогу враховувати відмінності в одиницях вимірювання та характері зміни телеметричних параметрів. Це дозволяє уникнути необхідності виконувати синхронізацію після кожного незначного коливання числових значень.

### 4.5. Інтеграція правил синхронізації в конвеєр оброблення

Розроблений компонент StateSynchronizer інтегровано в stream_receiver.py. Його екземпляр створюється під час ініціалізації компонентів конвеєра.

Після отримання повідомлення та успішного проходження попереднього оброблення виконується оновлення останнього достовірного значення сигналу. На основі збережених вимірювань формується поточний зріз телеметрії, який передається до методу check_message().

Цей метод перевіряє необхідність синхронізації за подією або за перевищенням порогу зміни числового показника.

Окремо в циклі роботи processing_worker() викликається метод check_periodic(), який контролює дотримання встановленого інтервалу періодичного оновлення.

У разі виконання будь-якого з правил викликається метод _sync(), який:

1. Формує структуру нового стану цифрового двійника.
2. Зберігає її в атрибуті current_state.
3. Оновлює збережені значення числових показників, використані для подальшого порівняння.
4. Збільшує лічильник виконаних синхронізацій.
5. Виводить інформацію про причину оновлення в консоль і записує результат до журналу.

Таким чином, синхронізація виконується на основі вже перевірених телеметричних даних, а не безпосередньо після отримання кожного MQTT-повідомлення.

### 4.6. Перевірка роботи правил синхронізації

Для перевірки реалізованих правил було запущено MQTT-брокер, емулятор електроскутера та оновлений компонент stream_receiver.py.

Під час роботи конвеєра було зафіксовано виконання синхронізації з різних причин. У консоль виводиться порядковий номер операції та її тип. Залежно від правила оновлення відображаються такі повідомлення:

- periodic – виконання періодичної синхронізації;
- event:SIG_BRK_07 або event:SIG_LCK_08 – синхронізація після зміни подієвого сигналу;
- threshold:SIG_VEL_01, threshold:SIG_TMP_05 тощо – синхронізація після перевищення порогу зміни відповідного числового показника.

Під час тестового запуску було отримано повідомлення про синхронізацію за порогом зміни швидкості, обертів двигуна та температури, а також періодичне оновлення стану. Приклад результату роботи компонента наведено на рисунку 4.1.

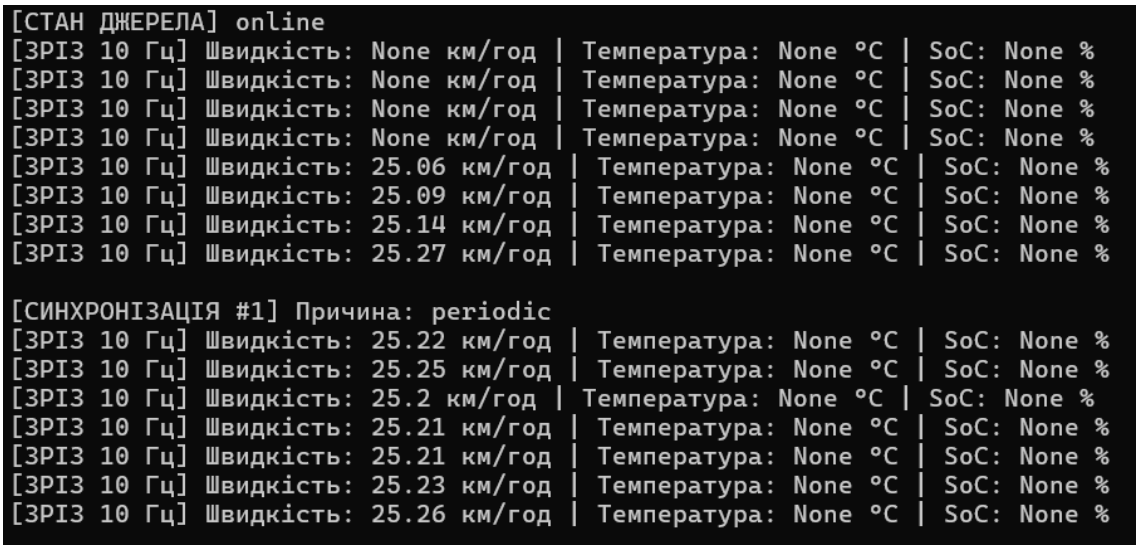



Рис. 4.1. Результат виконання правил синхронізації стану цифрового двійника.

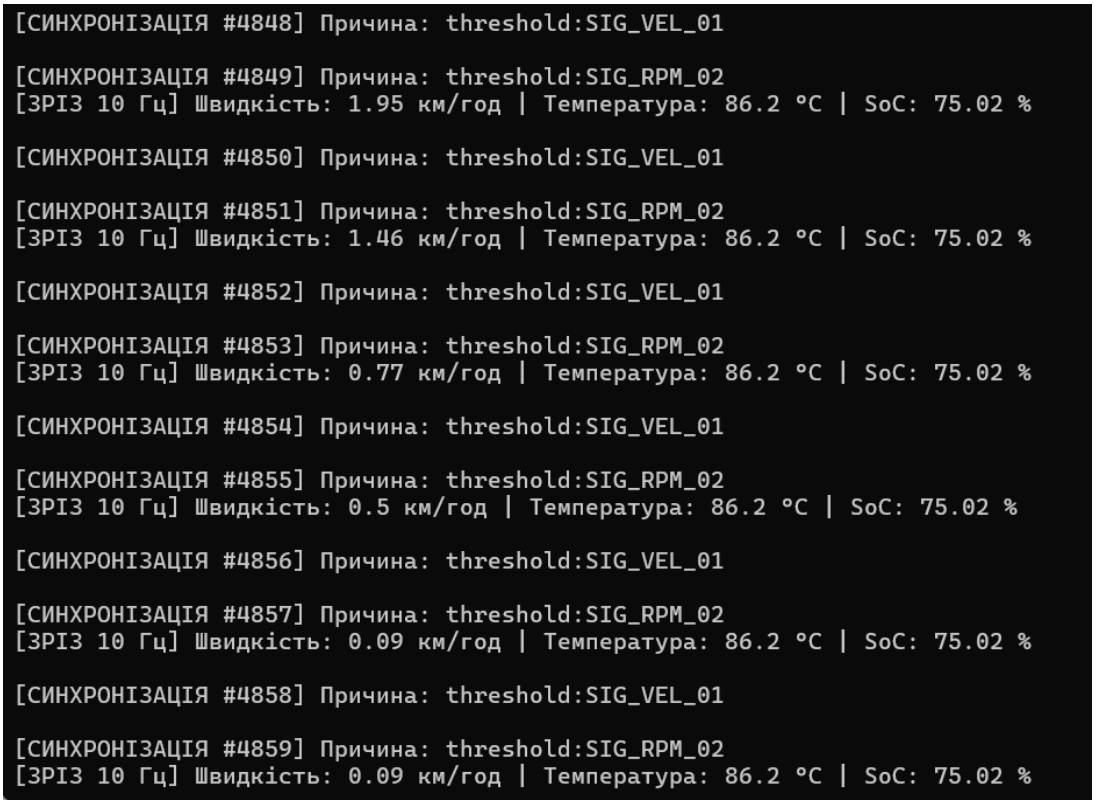



Рис. 4.2. Результат виконання правил синхронізації стану цифрового двійника.

Лічильник sync_count дає змогу відстежувати загальну кількість виконаних синхронізацій протягом поточного запуску програми. Причина кожного оновлення додатково зберігається в журналі stream_receiver.log разом із сформованою структурою стану.

Таким чином, у результаті виконання четвертого етапу реалізовано три правила синхронізації стану цифрового двійника електроскутера: періодичне оновлення з інтервалом 1 секунда, оновлення за зміною подієвих сигналів та оновлення за перевищенням порогів зміни числових показників. Розроблений компонент інтегровано в конвеєр оброблення телеметрії, що забезпечує оновлення стану цифрового двійника відповідно до характеру надходження та зміни даних.

## 5. Реалізувати компонент підтримання актуального стану двійника в пам’яті та механізм сповіщення підписників про зміну стану.

### 5.1. Реалізація компонента StateManager

Для підтримання актуального стану цифрового двійника в оперативній пам'яті розроблено окремий програмний компонент StateManager, реалізований у файлі state_manager.py.

Основним призначенням компонента є збереження останнього синхронізованого стану цифрового двійника та надання доступу до нього іншим компонентам системи.

Для цього реалізовано такі методи:

- update_state() – оновлення збереженого стану після виконання синхронізації;
- get_state() – отримання останнього збереженого стану;
- subscribe() – реєстрація підписника, який отримуватиме сповіщення про оновлення;
- unsubscribe() – скасування реєстрації підписника.

Поточний стан зберігається в атрибуті current_state. Для захисту даних від випадкової зміни зовнішніми компонентами під час передавання використовуються копії об'єктів. Додатково застосовується механізм блокування threading.RLock, який забезпечує узгоджений доступ до спільних даних у багатопотоковому середовищі.

### 5.2. Реалізація механізму сповіщення підписників

Для забезпечення передавання інформації про зміну стану цифрового двійника реалізовано механізм підписки на оновлення.

Під час ініціалізації конвеєра в stream_receiver.py створюється екземпляр StateManager, після чого реєструються два тестові підписники:

- interface – імітує отримання оновлень компонентом користувацького інтерфейсу;
- analytics – імітує отримання оновлень аналітичним компонентом.

Після виконання синхронізації компонент StateSynchronizer передає сформований стан до методу update_state() компонента StateManager. Після збереження нового стану відбувається виклик функцій усіх зареєстрованих підписників із передаванням їм копії оновленої структури даних.

Якщо під час виконання функції одного з підписників виникає помилка, вона фіксується в журналі, а оброблення інших підписників продовжується.

### 5.3. Інтеграція компонентів у конвеєр оброблення

Компонент StateManager інтегровано з раніше розробленим StateSynchronizer. Для цього під час створення екземпляра StateSynchronizer йому передається посилання на менеджер стану.

У результаті взаємодія компонентів відбувається за такою послідовністю:

Компонент StateSynchronizer визначає необхідність синхронізації відповідно до встановлених правил.

1) Метод _sync() формує нову структуру стану цифрового двійника.
2) Сформований стан передається до StateManager.
3) StateManager зберігає його в оперативній пам'яті.
4) Зареєстровані підписники отримують сповіщення про оновлення стану.

Таким чином, підтримання актуального стану та сповіщення зацікавлених компонентів винесено в окремий програмний модуль, що спрощує подальше розширення системи.

### 5.4. Перевірка роботи компонента

Для перевірки реалізованого функціоналу було запущено MQTT-брокер, емулятор електроскутера та оновлений компонент stream_receiver.py.

Після запуску конвеєра в консолі відображаються повідомлення про успішну реєстрацію двох підписників: interface та analytics.

Під час виконання синхронізації сформований стан передається до StateManager, після чого обидва підписники отримують відповідні

сповіщення. Результат їх виклику відображається в консолі повідомленнями:

*[SUBSCRIBER: INTERFACE] Отримано оновлення стану цифрового двійника.*

*[SUBSCRIBER: ANALYTICS] Отримано оновлення для аналізу телеметрії.*

Приклад результату роботи механізму сповіщення підписників наведено на рисунку 5.1.

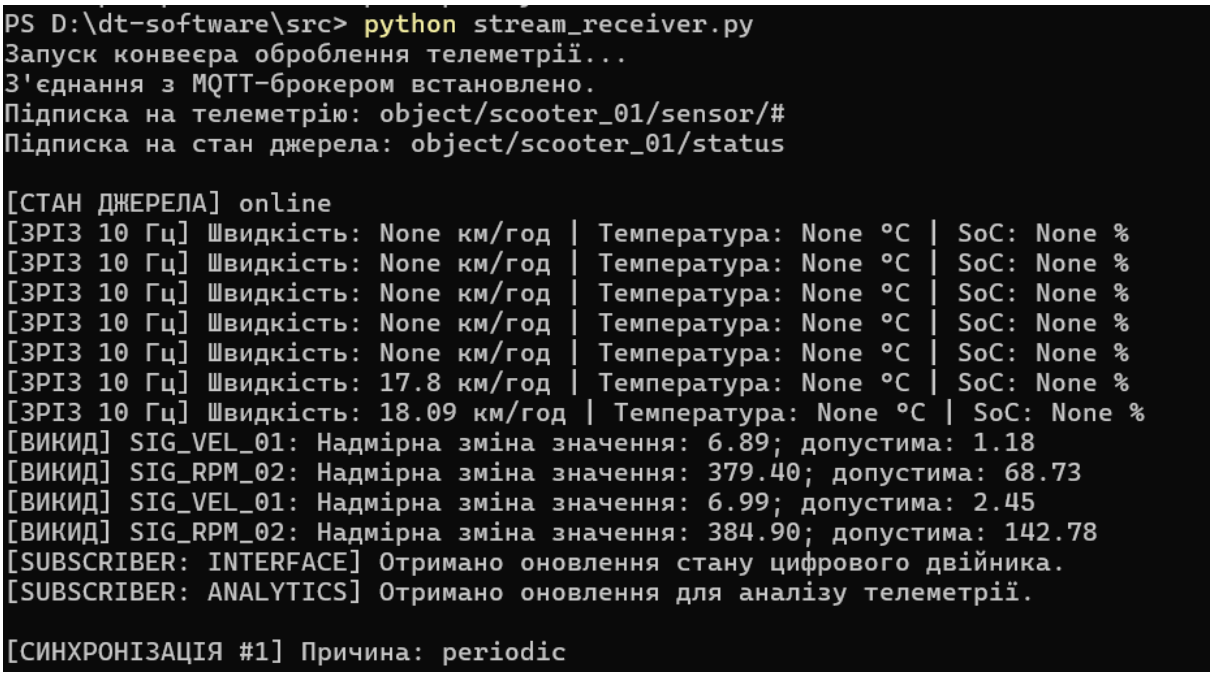



Рис. 5.1. Результат роботи механізму сповіщення підписників про оновлення стану цифрового двійника.

Таким чином, у результаті виконання п'ятого етапу розроблено компонент підтримання актуального стану цифрового двійника в оперативній пам'яті та механізм сповіщення зареєстрованих підписників про його зміну. Працездатність реалізації перевірено під час оброблення телеметрії, що надходить через MQTT-брокер.

**6. Розгорнути сховище даних часових рядів (наприклад, InfluxDB або TimescaleDB) та реалізувати збереження історії телеметрії і станів двійника.**

**6.1 Розгортання InfluxDB**

Для забезпечення довготривалого зберігання телеметрії та історії змін стану цифрового двійника необхідно розгорнути спеціалізоване сховище даних часових рядів. Для цього було обрано InfluxDB, яка призначена для зберігання та оброблення даних, пов'язаних із часовими мітками.

Для розгортання сховища використано Docker Desktop, завантажений з офіційного вебсайту Docker. Після встановлення було перевірено працездатність Docker Engine за допомогою команд docker --version, docker compose version та docker info. Результати перевірки підтвердили успішне встановлення Docker та готовність середовища до запуску контейнерів

```
PS D:\dt-software\src> docker --version
Docker version 29.9.2, build a82e528
PS D:\dt-software\src> docker compose version
Docker Compose version v5.6.1
PS D:\dt-software\src> docker info
Client:
 Version:    29.9.2
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.152.0
```

Рис. 6.1. Перевірка працездатність Docker Engine

Наступним кроком було підготовлено конфігурацію Docker Compose у файлі docker-compose.yml, яка визначає параметри розгортання InfluxDB у контейнері. Для збереження даних між перезапусками контейнера передбачено використання постійних томів Docker (volumes). Також створено файл .env для зберігання параметрів початкового налаштування сховища, зокрема облікових даних, назви організації та сховища даних (bucket).

Для запобігання випадковому оприлюдненню конфіденційних параметрів доступу файл .env додано до .gitignore. Це забезпечує виключення облікових даних і токена доступу до InfluxDB з переліку файлів, які відстежуються системою контролю версій Git.

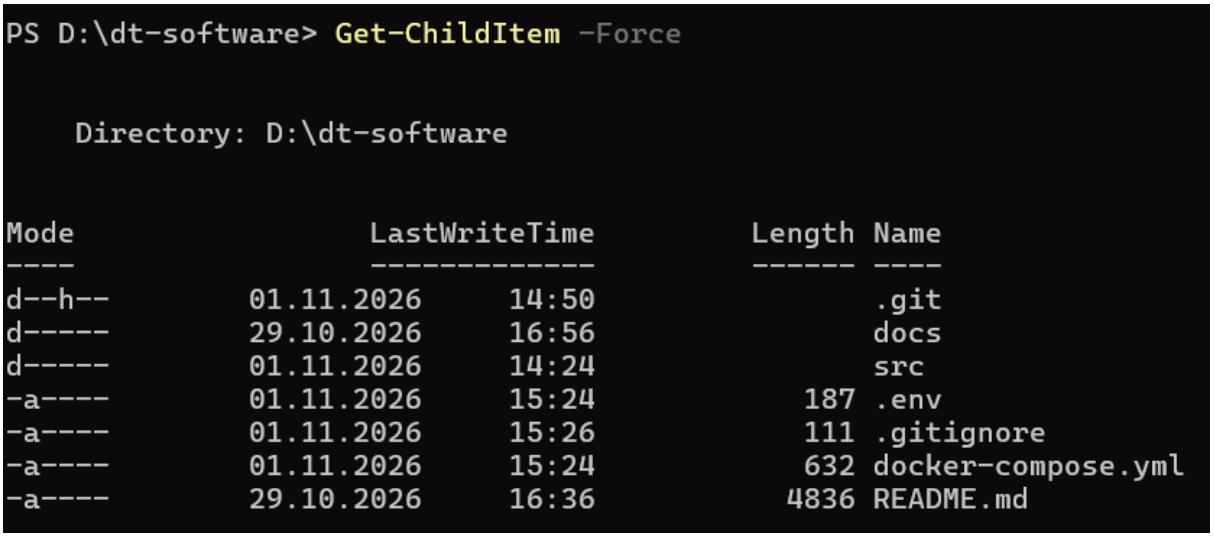



Рис. 6.2. Вміст кореневої папки

Видно, що файли docker-compose.yml та .env присутні. Отже, після підготовки конфігураційних файлів було виконано розгортання InfluxDB за допомогою Docker Compose. Для цього використано команду docker compose up -d, яка забезпечує завантаження необхідного образу, створення контейнера та запуск сховища у фоновому режимі. Після запуску перевірено стан контейнера за допомогою команди docker compose ps. Результат перевірки підтвердив успішний запуск InfluxDB.

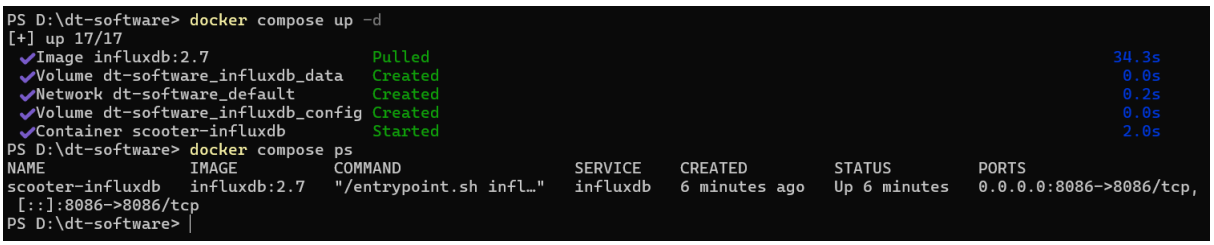



Рис. 6.3. Результат розгортання та перевірки стану контейнера InfluxDB.

Після запуску контейнера було відкрито вебінтерфейс InfluxDB за адресою http://localhost:8086. Під час початкового налаштування використано параметри, визначені у файлі .env, зокрема назву організації dt-org та сховища даних scooter_telemetry.

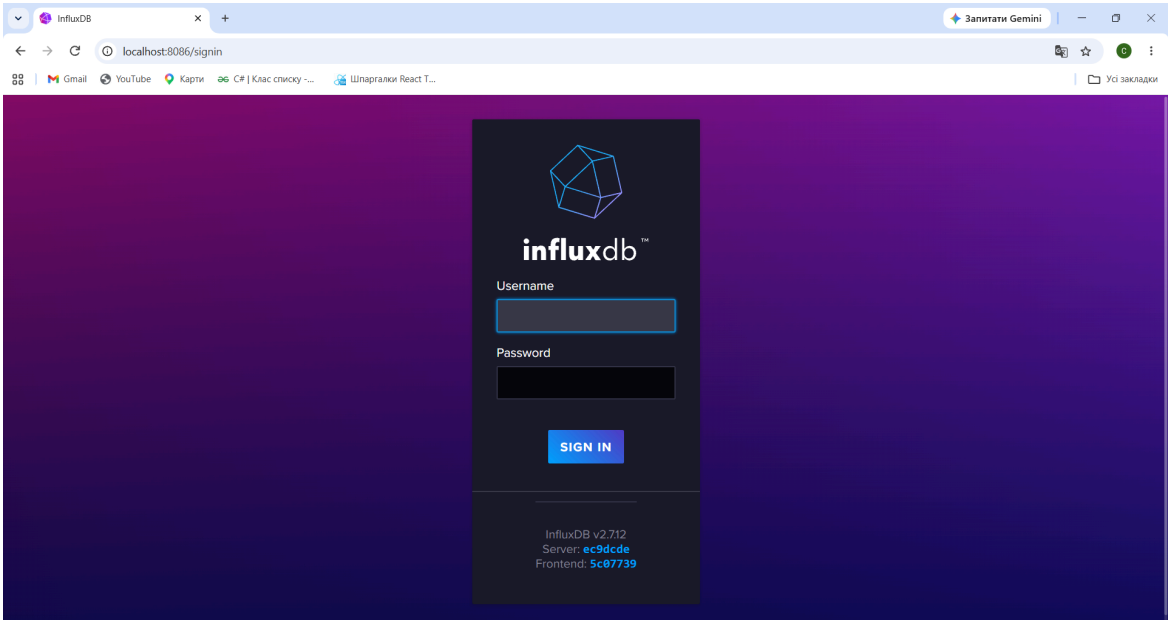



Рис. 6.4. Сторінка реєстрації InfluxDB

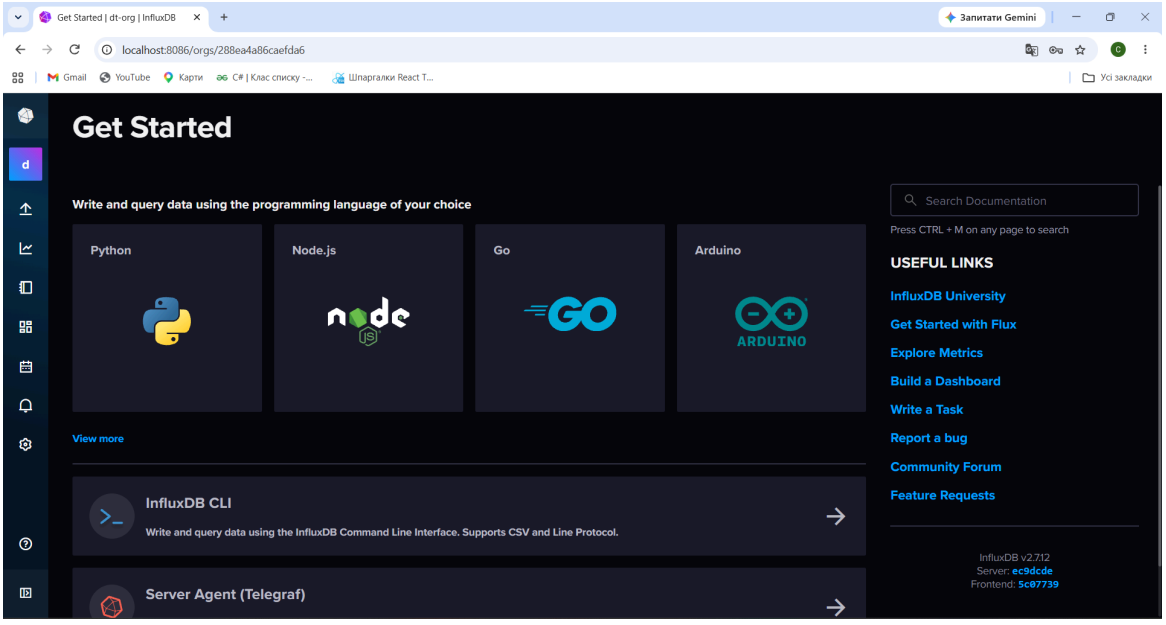



Рис. 6.5. Головна сторінка вебінтерфейсу InfluxDB

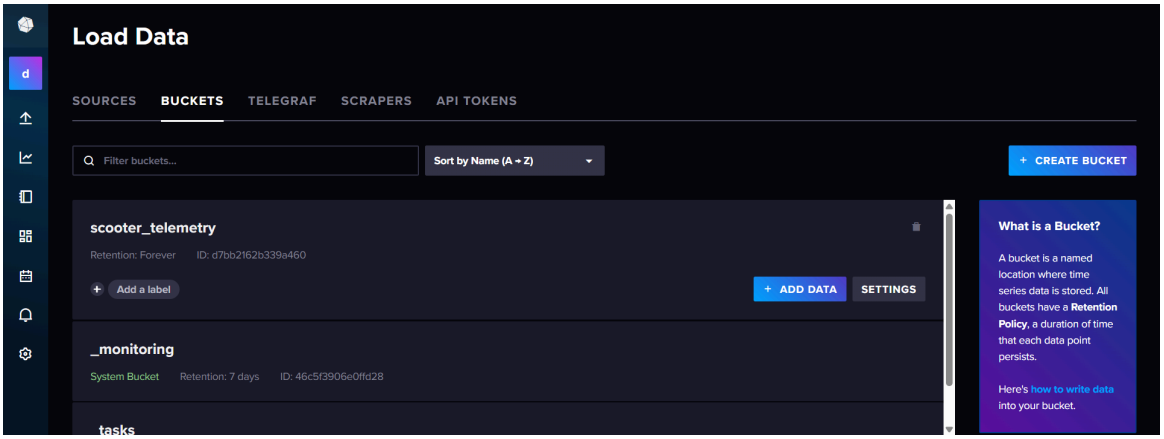



Рис. 6.6. Перелік створених сховищ даних (Buckets)

### 6.2. Реалізація збереження телеметрії та станів цифрового двійника

Для забезпечення взаємодії розробленого конвеєра оброблення телеметрії зі сховищем даних було створено окремий програмний компонент database_manager.py. Його основним призначенням є організація взаємодії з InfluxDB, збереження отриманих вимірювань датчиків та формування історії змін стану цифрового двійника.

Для реалізації взаємодії з базою даних використано бібліотеку influxdb-client, яка забезпечує підключення до InfluxDB, формування записів та виконання операцій запису. Завантаження конфігураційних параметрів здійснюється за допомогою бібліотеки python-dotenv, що дає змогу отримувати адресу сховища, токен доступу, назву організації та сховища даних із файлу .env без безпосереднього зазначення цих параметрів у програмному коді.

У класі DatabaseManager реалізовано чотири основні методи:

- connect() – перевіряє наявність необхідних параметрів конфігурації, створює клієнт InfluxDB та ініціалізує інтерфейс для запису даних. Для виконання операцій запису використовується синхронний режим, за якого програма очікує завершення відповідної операції.
- write_telemetry(data, timestamp) – забезпечує збереження окремих вимірювань датчиків. Для кожного запису формується точка з назвою вимірювання telemetry, ідентифікатором електроскутера, ідентифікатором сенсора, його назвою та одиницею вимірювання. Часова мітка визначає момент отримання вимірювання відповідно до даних датчика.
- write_twin_state(state) – відповідає за збереження сформованих знімків стану цифрового двійника. Дані записуються до окремого вимірювання twin_state. Структура стану серіалізується у формат JSON та зберігається разом із часовою міткою і причиною синхронізації.
- close() – закриває інтерфейс запису та з'єднання з InfluxDB після завершення роботи конвеєра.

Під час формування записів телеметрії враховано різні типи даних, які надходять від датчиків електроскутера. Для числових вимірювань використовується поле value, тоді як логічні значення стану гальмування та протиугінного блокування записуються в окреме поле value_bool. Такий підхід дає змогу уникнути конфліктів типів даних під час збереження вимірювань у межах одного сховища.

Для збереження телеметрії було модифіковано компонент stream_receiver.py. До нього додано імпорт екземпляра database_manager, а також виклик методу write_telemetry() після проходження повідомленням перевірки коректності та фільтрації аномальних значень. Таким чином, до сховища передаються лише прийняті вимірювання, які пройшли попереднє оброблення. Запізнілі, некоректні та відхилені як аномальні повідомлення не записуються до історії телеметрії.

Окремо реалізовано збереження історії станів цифрового двійника. Для цього використано вже наявний механізм підписки компонента StateManager. Під час запуску конвеєра до нього реєструється додатковий підписник із назвою database, якому передається метод write_twin_state(). У результаті після кожної синхронізації стану, незалежно від того, чи була вона викликана часовим інтервалом, зміною значення датчика або подією, сформований знімок автоматично передається до компонента збереження.

Також у функції main() додано встановлення з'єднання з InfluxDB перед запуском потоку оброблення повідомлень. Після завершення роботи конвеєра передбачено закриття з'єднання зі сховищем. Це забезпечує узгоджене керування життєвим циклом підключення до бази даних.

Отже, у результаті реалізації було забезпечено два незалежні напрями збереження інформації: історію окремих вимірювань датчиків та історію синхронізованих станів цифрового двійника. Для їх зберігання використовуються окремі вимірювання telemetry і twin_state, що дає змогу надалі виконувати вибірку та аналіз даних за часовими мітками.

**6.3. Запуск і перевірка працездатності конвеєра зі сховищем даних**

Після реалізації програмного компонента database_manager.py та його інтеграції з stream_receiver.py було виконано перевірку взаємодії конвеєра оброблення телеметрії зі сховищем InfluxDB.

На першому етапі перевірено працездатність контейнера InfluxDB за допомогою команди:

*docker compose ps*

Результат виконання команди підтвердив, що контейнер scooter-influxdb перебуває в робочому стані.

Далі було перевірено наявність необхідних Python-бібліотек influxdb-client та python-dotenv, які використовуються для взаємодії зі сховищем і завантаження конфігураційних параметрів.

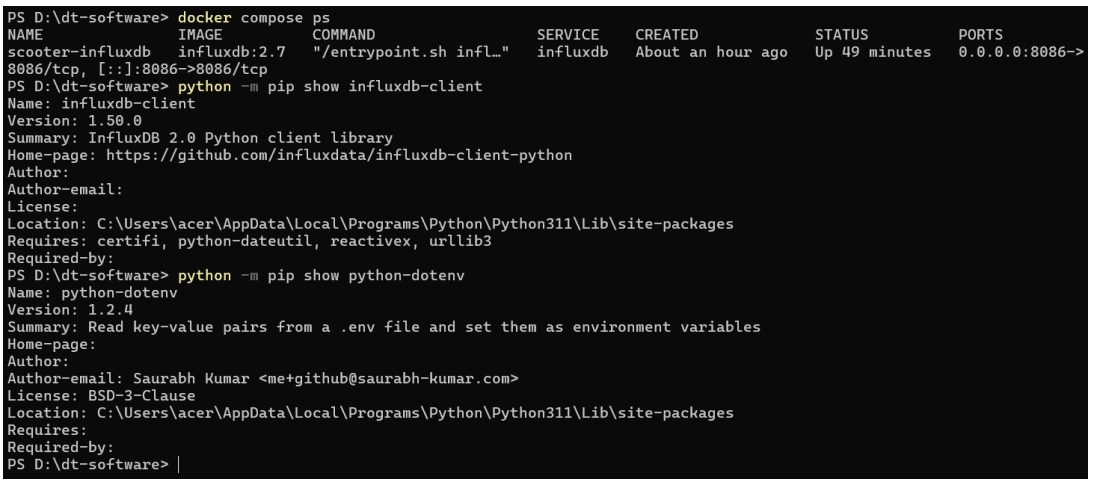



Рис. 6.7. перевірено працездатності контейнера InfluxDB та Python-бібліотек

Після підготовки середовища виконано перевірку встановлення з'єднання між Python-компонентом та InfluxDB. Для цього використано команду:

*python -c "from database_manager import database_manager; database_manager.connect(); database_manager.close()"*

У разі успішного виконання команди в консолі відображається повідомлення про встановлення з'єднання з InfluxDB.

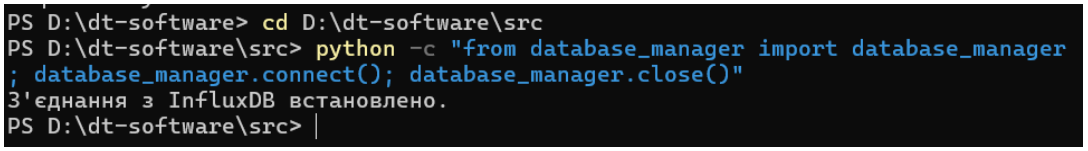



Рис. 6.8. Перевірка встановлення з'єднання між Python-компонентом та InfluxDB

Після перевірки підключення було виконано запуск компонентів системи. Спочатку забезпечено роботу MQTT-брокера, після чого запущено компонент stream_receiver.py, який встановлює з'єднання з InfluxDB, реєструє підписників та розпочинає оброблення вхідних повідомлень. Наступним було запущено емулятор електроскутера emulator_publisher.py, який формує та передає телеметричні дані через MQTT-брокер.

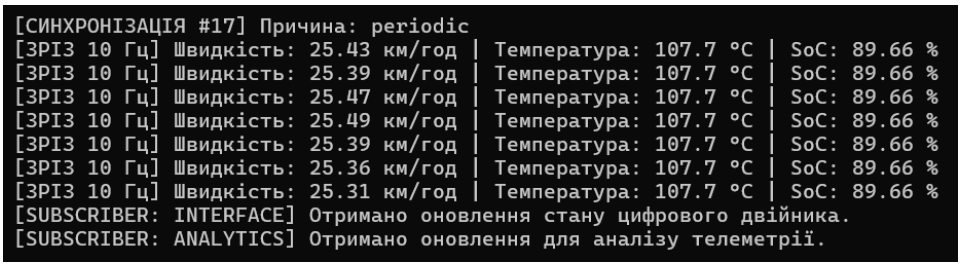



Рис. 6.9. Результат запуску stream_receiver.py

Під час роботи системи отримані повідомлення проходять етапи впорядкування, валідації та фільтрації аномальних значень. Прийняті вимірювання передаються до компонента database_manager.py для запису в InfluxDB. Одночасно компонент StateSynchronizer формує оновлення стану цифрового двійника, які через StateManager передаються зареєстрованим підписникам, зокрема компоненту збереження даних.

Таким чином, під час виконання конвеєра забезпечується паралельне оновлення актуального стану цифрового двійника та накопичення історичних даних у часовому сховищі.

### 6.4. Перевірка збереження телеметрії та станів цифрового двійника

Для перевірки результатів збереження даних було використано вебінтерфейс InfluxDB. У розділі Data Explorer вибрано сховище scooter_telemetry, після чого виконано перегляд записів за відповідними вимірюваннями.

Для перевірки історії телеметрії було досліджено вимірювання telemetry, яке містить дані окремих датчиків електроскутера. Записи містять ідентифікатори сенсорів, назви показників, одиниці вимірювання, значення та часові мітки. Це дає змогу відстежувати зміну фізичних параметрів електроскутера в часі.




Рис. 6.10. Перевірка історії телеметрії у веб інтерфейсі InfluxDB

Окремо перевірено вимірювання twin_state, призначене для збереження історії синхронізованих станів цифрового двійника. Кожен запис містить JSON-представлення стану, часову мітку його формування та причину синхронізації.

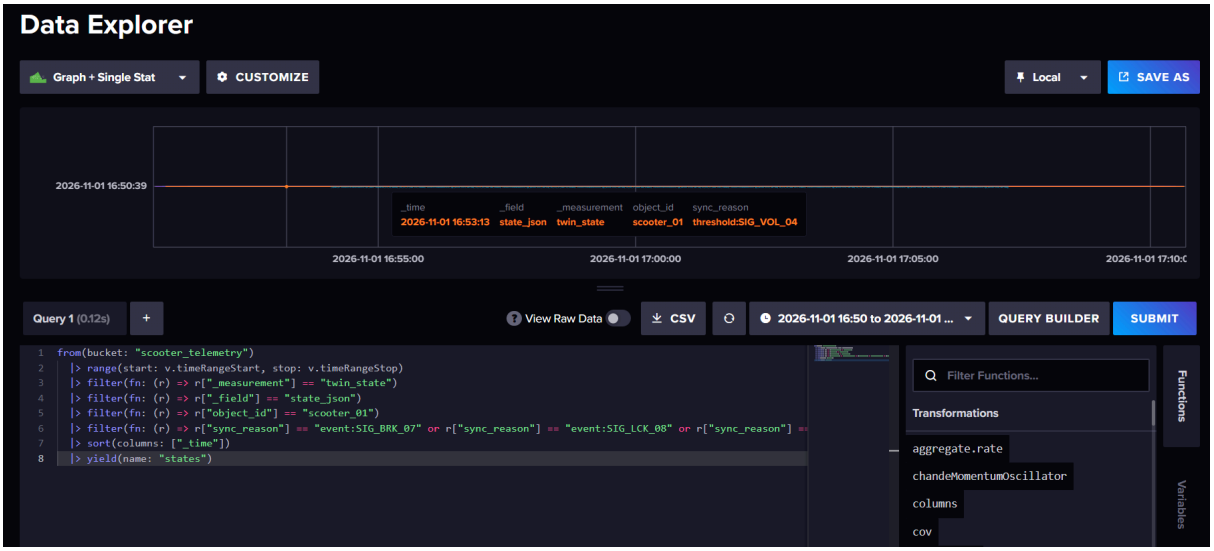



Рис. 6.11. Перевірка вимірювання twin_state у веб інтерфейсі InfluxDB

Результати перевірки підтверджують можливість накопичення історії телеметричних вимірювань і змін стану цифрового двійника в InfluxDB.

**7. Виміряти характеристики синхронізації: середню та максимальну затримку між моментом генерації вимірювання та оновленням стану двійника, пропускну здатність конвеєра.**

Для оцінювання продуктивності розробленого конвеєра оброблення потокових даних було реалізовано окремий програмний компонент PerformanceMonitor, призначений для вимірювання затримки синхронізації стану цифрового двійника та пропускної здатності системи.

Метою цього етапу є визначення часу, необхідного для проходження телеметричного вимірювання від моменту його генерації емулятором до моменту оновлення стану цифрового двійника, а також оцінювання кількості повідомлень, які конвеєр здатний обробити за одиницю часу.

### 7.1. Реалізація компонента вимірювання продуктивності

Для збору характеристик продуктивності створено програмний модуль performance_monitor.py, який містить клас PerformanceMonitor.

У компоненті реалізовано такі основні методи:

- start() – розпочинає вимірювання, фіксує початковий момент часу та обнуляє лічильники;
- record_processed_message() – реєструє кожне успішно оброблене телеметричне повідомлення;

- record_state_update() – обчислює затримку між генерацією вимірювання та оновленням стану цифрового двійника;
- stop() – завершує вимірювання, розраховує підсумкові показники та виводить результати.

Для визначення тривалості вимірювання та підрахунку пропускної здатності використовується функція time.monotonic(). Вона забезпечує вимірювання інтервалів часу незалежно від можливих змін системного годинника.

Для обчислення затримки використовуються часові мітки, які вже формуються емулятором сенсорів та зберігаються в моделі стану цифрового двійника. Момент генерації вимірювання визначається за атрибутом timestamp, а момент фактичного оновлення стану фіксується в компоненті StateManager.

Затримка окремого вимірювання визначається за формулою:

$$L_i = t_{update,i} - t_{generation,i}$$

де:

- $L_i$ – затримка оброблення вимірювання, мс;
- $t_{update,i}$ – момент оновлення стану цифрового двійника;
- $t_{generation,i}$ – момент генерації вимірювання сенсором.

За результатами зібраних значень визначаються середня та максимальна затримки:

$$L_{avg} = \frac{1}{N}\sum_{i=1}^{N} L_i$$

$$L_{max} = max(L_1, L_2, \ldots, L_N)$$

де $N$ – кількість вимірювань, для яких вдалося розрахувати затримку.

Для уникнення повторного врахування тих самих вимірювань у компоненті використовується множина seen_measurements. Вона дає змогу ідентифікувати кожне вимірювання за ідентифікатором сенсора та часовою міткою його генерації.

Пропускна здатність конвеєра визначається як відношення кількості успішно оброблених повідомлень до тривалості вимірювання:

$$Q = \frac{N_{processed}}{T}$$

де:

- $Q$ – пропускна здатність, повідомлень/с;

- $N_{processed}$ – кількість успішно оброблених повідомлень;
- $T$ – тривалість вимірювання, с.

### 7.2. Інтеграція засобів вимірювання в конвеєр

Компонент PerformanceMonitor інтегровано з раніше розробленими модулями stream_receiver.py та state_manager.py.

У StateManager передбачено передавання посилання на екземпляр монітора під час його ініціалізації. Після оновлення поточного стану цифрового двійника компонент фіксує фактичний момент оновлення та передає його разом зі сформованою структурою стану до методу record_state_update().

Це дає змогу вимірювати затримку безпосередньо в момент оновлення стану, не включаючи до неї час подальшого виконання функцій підписників, зокрема запису до InfluxDB.

У компоненті stream_receiver.py додатково реалізовано параметр командного рядка --measure, який дає змогу запускати конвеєр у спеціальному режимі вимірювання продуктивності.

Передбачено два варіанти запуску:

- *python stream_receiver.py* – звичайний режим роботи конвеєра без збору показників продуктивності.
- *python stream_receiver.py --measure* – режим із додатковим вимірюванням характеристик синхронізації.

Під час оброблення повідомлень викликається метод record_processed_message(), який збільшує лічильник успішно оброблених повідомлень. Водночас вимірювання затримки здійснюється під час оновлення стану в StateManager.

Таким чином, збір показників продуктивності не змінює реалізованих правил синхронізації та може бути ввімкнений лише за потреби проведення тестування.

### 7.3. Проведення вимірювання продуктивності

Для перевірки характеристик конвеєра було використано штатний сценарій роботи електроскутера з емуляцією телеметричних сигналів різної частоти.

Перед проведенням тестування було запущено MQTT-брокер Mosquitto та контейнер InfluxDB. Після цього в окремих вікнах PowerShell запущено компонент приймання потокових даних і емулятор електроскутера.

Для активації збору показників використано команду:

*python stream_receiver.py --measure*

Після встановлення з'єднання з MQTT-брокером та запуску вимірювання емулятор почав передавати телеметричні повідомлення. Конвеєр виконував їх буферизацію, валідацію, фільтрацію, оновлення актуального стану та збереження даних у InfluxDB.

Після завершення тестового запуску компонент PerformanceMonitor формує підсумковий звіт, який містить тривалість вимірювання, кількість оброблених повідомлень, пропускну здатність, кількість вимірювань із розрахованою затримкою, а також середню та максимальну затримки.

### 7.4. Результати вимірювання

```
[СИНХРОНІЗАЦІЯ #122] Причина: periodic
[ЗРІЗ 10 Гц] Швидкість: 2.46 км/год | Температура: 64.7 °C | SoC: 88.68 %
[ЗРІЗ 10 Гц] Швидкість: 2.98 км/год | Температура: 64.7 °C | SoC: 88.68 %
[ЗРІЗ 10 Гц] Швидкість: 3.07 км/год | Температура: 64.7 °C | SoC: 88.68 %
[ЗРІЗ 10 Гц] Швидкість: 3.31 км/год | Температура: 64.7 °C | SoC: 88.68 %
[ЗРІЗ 10 Гц] Швидкість: 3.79 км/год | Температура: 64.7 °C | SoC: 88.68 %
[ЗРІЗ 10 Гц] Швидкість: 3.98 км/год | Температура: 64.7 °C | SoC: 88.68 %
[ЗРІЗ 10 Гц] Швидкість: 4.16 км/год | Температура: 64.7 °C | SoC: 88.68 %

======================================================
РЕЗУЛЬТАТИ ВИМІРЮВАННЯ ПРОДУКТИВНОСТІ
======================================================
Тривалість тесту: 77.02 с
Оброблено повідомлень: 1412
Пропускна здатність: 18.33 повідомлень/с
Кількість вимірювань із розрахованою затримкою: 47
Середня затримка: 4022.109 мс
Максимальна затримка: 12518.838 мс
======================================================

Конвеєр оброблення завершив роботу.
PS D:\dt-software\src>
```

Рис. 7.1. Результат вимірювання продуктивності конвеєра оброблення потокових даних.

За результатами проведеного тестування можна оцінити швидкість оброблення телеметричних повідомлень та час оновлення цифрового двійника відносно моменту генерації вимірювань. Отримані показники дають змогу кількісно оцінити роботу реалізованих механізмів буферизації, попереднього оброблення та синхронізації.

Таким чином, у результаті виконання сьомого етапу розроблено та інтегровано засоби вимірювання продуктивності конвеєра. Реалізація забезпечує визначення середньої та максимальної затримки синхронізації,

а також пропускної здатності системи. Отримані результати можуть використовуватися для подальшого оцінювання роботи конвеєра за різних сценаріїв навантаження.

**8. Перевірити роботу конвеєра за різних сценаріїв (штатний режим, сплеск інтенсивності даних, тимчасова відсутність зв’язку); оформити звіт.**

**8.1. Перевірка штатного режиму**

Для перевірки роботи конвеєра за штатного режиму було запущено MQTT-брокер Mosquitto, контейнер InfluxDB, програмний компонент приймання телеметрії та емулятор сенсорів електроскутера.

Спочатку було запущено MQTT-брокер Mosquitto та контейнер InfluxDB. Після цього в першому терміналі запущено компонент приймання й оброблення потокових даних командою:

*python stream_receiver.py*

У другому терміналі запущено публікатор телеметрії:

*python emulator_publisher.py*

Після встановлення MQTT-з'єднання емулятор розпочав формування та передавання повідомлень відповідно до заданих частот сенсорів: 10 Гц, 1 Гц, 0,1 Гц, а також під час зміни дискретних станів. Отримані повідомлення проходили попереднє оброблення, після чого використовувалися для оновлення стану цифрового двійника.

На рисунку 8.1 наведено процес запуску компонентів системи та надходження телеметрії в штатному режимі.

```
PS D:\dt-software\src> python stream_receiver.py
Запуск конвеєра оброблення телеметрії...
Режим: звичайна робота.
З'єднання з InfluxDB встановлено.
З'єднання з MQTT-брокером встановлено.
Підписка на телеметрію: object/scooter_01/sensor/#
Підписка на стан джерела: object/scooter_01/status

[СТАН ДЖЕРЕЛА] online
[ЗРІЗ 10 Гц] Швидкість: None км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: None км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: None км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: None км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: 6.51 км/год | Температура: None °C | SoC: None %
[SUBSCRIBER: INTERFACE] Отримано оновлення стану цифрового двійника.
[SUBSCRIBER: ANALYTICS] Отримано оновлення для аналізу телеметрії.

[СИНХРОНІЗАЦІЯ #1] Причина: periodic
[ЗРІЗ 10 Гц] Швидкість: 6.76 км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: 7.39 км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: 8.03 км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: 8.17 км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: 8.31 км/год | Температура: None °C | SoC: None %
[ЗРІЗ 10 Гц] Швидкість: 8.52 км/год | Температура: None °C | SoC: None %
[SUBSCRIBER: INTERFACE] Отримано оновлення стану цифрового двійника.
```

Рис. 8.1 Запуск компонентів конвеєра та публікація телеметрії у штатному режимі

Під час перевірки контролювалася поява MQTT-повідомлень, виконання синхронізації стану цифрового двійника та відсутність критичних помилок під час оброблення даних. Додатково перевірялася наявність актуальних вимірювань у базі даних InfluxDB за допомогою Data Explorer.

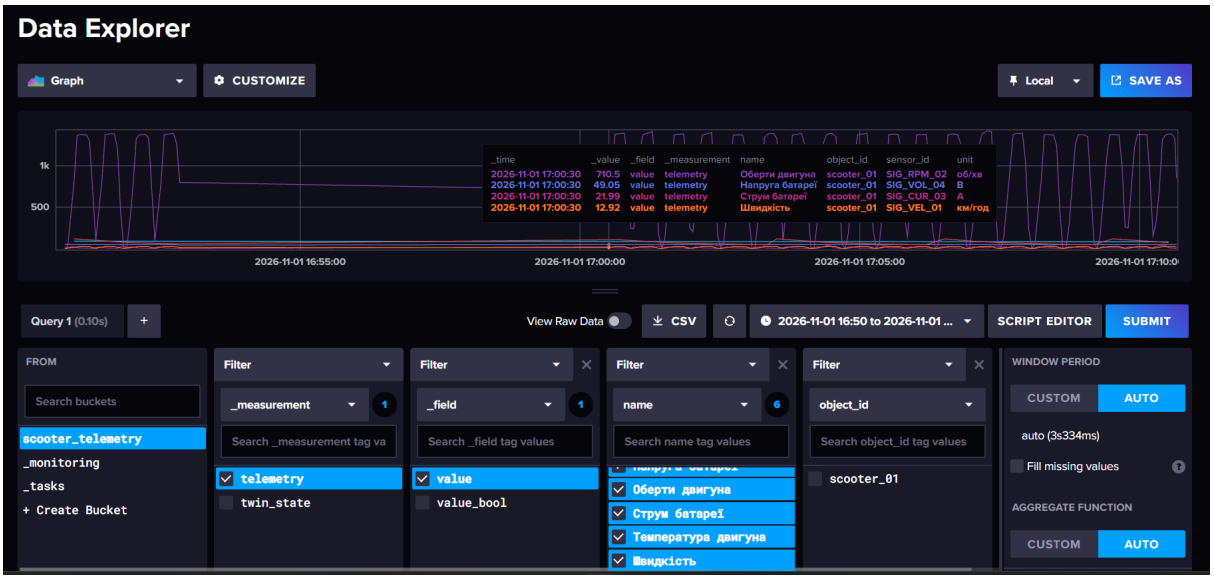



Рис. 8.2. Перегляд збережених вимірювань телеметрії в InfluxDBЗа результатами перевірки було підтверджено

За результатами перевірки підтверджено, що система забезпечує приймання телеметрії від емулятора через MQTT-брокер, попереднє оброблення отриманих повідомлень і синхронізацію стану цифрового двійника. Також підтверджено збереження вимірювань у базі даних InfluxDB та можливість їх перегляду за допомогою Data Explorer. Під час роботи у штатному режимі критичних помилок, що перешкоджали обробленню телеметрії, не виявлено.

### 8.2. Порівняння сценаріїв роботи конвеєра

Для порівняння роботи конвеєра було проведено три окремі експерименти: у штатному режимі, за умов сплеску інтенсивності надходження даних та під час тимчасової відсутності зв'язку з MQTT-брокером.

Для кожного експерименту компонент приймання й оброблення потокових даних запускався з однаковими параметрами:

*python stream_receiver.py --measure --duration 60*

Для емулятора сенсорів використовувалися такі команди залежно від сценарію:

Сценарій 1 – штатний режим:

*python emulator_publisher.py*

Сценарій 2 – сплеск інтенсивності даних:

*python emulator_publisher.py --scenario burst --scenario-start 10 --scenario-duration 5 --burst-factor 5*

Сценарій 3 – тимчасова відсутність зв'язку:

*python emulator_publisher.py --scenario disconnect --scenario-start 10 --scenario-duration 5*

Під час кожного експерименту фіксувалися тривалість роботи конвеєра, кількість отриманих та оброблених повідомлень, пропускна здатність, а також середня й максимальна затримки оброблення.

Результати першого експерименту наведено на рисунку 8.3.

```
============================================================
РЕЗУЛЬТАТИ ВИМІРЮВАННЯ ПРОДУКТИВНОСТІ
============================================================
Тривалість тесту: 70.95 с
Оброблено повідомлень: 1317
Пропускна здатність: 18.56 повідомлень/с
Кількість вимірювань із розрахованою затримкою: 33
Середня затримка: 3867.583 мс
Максимальна затримка: 11387.884 мс
============================================================


============================================================
СТАТИСТИКА ПІДПИСНИКА
============================================================
Отримано повідомлень: 1337
Оброблено повідомлень: 1317
============================================================

Конвеєр оброблення завершив роботу.
PS D:\dt-software\src>
```

Рис. 8.3. Результати вимірювання продуктивності конвеєра у штатному режимі.

Результати другого експерименту наведено на рисунку 8.4.

```
============================================================
РЕЗУЛЬТАТИ ВИМІРЮВАННЯ ПРОДУКТИВНОСТІ
============================================================
Тривалість тесту: 82.28 с
Оброблено повідомлень: 1509
Пропускна здатність: 18.34 повідомлень/с
Кількість вимірювань із розрахованою затримкою: 44
Середня затримка: 8403.746 мс
Максимальна затримка: 22677.136 мс
============================================================


============================================================
СТАТИСТИКА ПІДПИСНИКА
============================================================
Отримано повідомлень: 1784
Оброблено повідомлень: 1509
============================================================

Конвеєр оброблення завершив роботу.
PS D:\dt-software\src>
```

Рис. 8.4. Результати вимірювання продуктивності конвеєра за умов сплеску інтенсивності даних.

Результати третього експерименту наведено на рисунку 8.5.

```
============================================================
РЕЗУЛЬТАТИ ВИМІРЮВАННЯ ПРОДУКТИВНОСТІ
============================================================
Тривалість тесту: 70.19 с
Оброблено повідомлень: 1218
Пропускна здатність: 17.35 повідомлень/с
Кількість вимірювань із розрахованою затримкою: 50
Середня затримка: 2926.579 мс
Максимальна затримка: 10165.002 мс
============================================================


============================================================
СТАТИСТИКА ПІДПИСНИКА
============================================================
Отримано повідомлень: 1228
Оброблено повідомлень: 1218
============================================================

Конвеєр оброблення завершив роботу.
PS D:\dt-software\src>
```

Рис. 8.5. Результати вимірювання продуктивності конвеєра під час тимчасової відсутності зв'язку.

Таблиця 8.1.

Порівняння продуктивності конвеєра за різних сценаріїв

| Показник | Штатний режим | Сплеск інтенсивності | Відсутність зв'язку |
|---|---|---|---|
| Тривалість тесту, с | 70,95 | 82,28 | 70,19 |
| Отримано повідомлень | 1337 | 1784 | 1228 |
| Оброблено повідомлень | 1317 | 1509 | 1218 |
| Пропускна здатність, повід./с | 18,56 | 18,34 | 17,35 |
| Середня затримка, мс | 3867,583 | 8403,746 | 2926,579 |
| Максимальна затримка, мс | 11387,884 | 22677,136 | 10165,002 |
| Вимірювань із розрахованою затримкою | 33 | 44 | 50 |

За результатами порівняння встановлено, що у штатному режимі конвеєр обробив 1317 із 1337 отриманих повідомлень із пропускною здатністю 18,56 повідомлення/с.

Під час сплеску інтенсивності кількість отриманих повідомлень збільшилася до 1784, а кількість оброблених – до 1509. Водночас середня затримка зросла до 8403,746 мс, а максимальна – до 22677,136 мс. Пропускна здатність становила 18,34 повідомлення/с. Це свідчить про збільшення навантаження на конвеєр та зростання затримок під час інтенсивнішого надходження телеметрії.

Під час тимчасової відсутності зв'язку було отримано 1228 повідомлень, з яких оброблено 1218. Пропускна здатність становила 17,35 повідомлення/с. Після відновлення з'єднання передавання телеметрії продовжилося. Середня затримка становила 2926,579 мс, а максимальна – 10165,002 мс. Отже, конвеєр продовжив роботу після відновлення зв'язку, проте повідомлення, які не були передані під час перерви, не відновлювалися.

В усіх трьох експериментах кількість оброблених повідомлень була меншою за кількість отриманих. Це свідчить про наявність повідомлень, які не були враховані як оброблені до завершення вимірювання. Найбільша різниця спостерігалася під час сплеску інтенсивності та становила 275 повідомлень.

Таким чином, проведені експерименти дали змогу оцінити роботу конвеєра за різних умов надходження телеметрії. Найбільше зростання затримок спостерігалося під час сплеску інтенсивності, тоді як після тимчасової втрати зв'язку передавання даних відновлювалося.

**Висновок**

У ході виконання комп’ютерного практикуму було успішно розроблено та повноцінно протестовано програмний конвеєр для оброблення потокових даних у реальному часі та синхронізації стану цифрового двійника електроскутера (scooter_01).

Спершу було спроєктовано багаторівневу модель стану цифрового двійника, яка поєднує первинні телеметричні сигнали, похідні показники та службові метадані. Для приймання та буферизації повідомлень було задіяно IoT-канал через MQTT із впорядкуванням за часовими мітками за допомогою мін-купи (heapq). Також реалізовано комплексне попереднє оброблення даних, що охоплює їх валідацію, фільтрацію аномальних викидів за швидкістю зміни, контроль актуальності (пропусків) та ресемплінг із єдиною цільовою частотою 10 Гц. Для оптимізації навантаження впроваджено комбіновані правила синхронізації стану, а саме періодичне оновлення, оновлення за подією та за порогом зміни значення.

Окрім цього, було створено менеджер стану в оперативній пам'яті (StateManager) із захистом багатопотоковості та гібридним механізмом сповіщення підписників за патерном Pub/Sub. Для довготривалого зберігання даних розгорнуто часове сховище InfluxDB за допомогою Docker Desktop, що забезпечило паралельне збереження сирої історії телеметрії (telemetry) та знімків стану двійника (twin_state). За допомогою компонента PerformanceMonitor було виміряно характеристики затримки й пропускної здатності конвеєра та протестовано його роботу за трьома основними сценаріями: у штатному режимі, під час сплеску інтенсивності даних та в умовах тимчасової відсутності зв'язку.

Отримані результати підтвердили стійкість і працездатність розробленої архітектури, а всі програмні компоненти та звіт повністю оформлено відповідно до академічних вимог.